Banking Complaint Intake and Compliance-Routing Agent

1. Use Case and Why It Matters for a Bank

The prototype handles end-to-end intake and routing of customer complaints. It validates the customer ID and complaint text, classifies the complaint into one of six categories - account access, billing dispute, compliance violation, fraud, poor service, or service delay - checks policy and completeness requirements, selects the next action, creates or reuses a ticket, generates an acknowledgement, and records an execution trace.

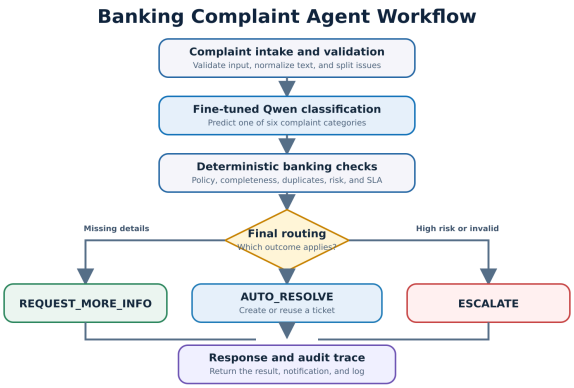
This matters because complaint handling is operationally expensive and compliance-sensitive. Consistent classification and routing can reduce manual triage, direct urgent fraud and compliance cases to human reviewers sooner, prevent avoidable duplicate tickets, and give staff a transparent record of how each decision was reached. The design automates repeatable steps without allowing the language model to make final compliance decisions by itself.

2. Agent Architecture and Design

The agent uses a deterministic LangGraph StateGraph. Its main path is: intake validation -> text normalization and multi-issue detection -> Qwen classification -> output-validity gate -> policy, history, completeness, duplicate and risk checks -> decision and routing -> ticketing, notification, acknowledgement and audit logging. Conditional edges produce three final outcomes: AUTO_RESOLVE, REQUEST_MORE_INFO, or ESCALATE.

The QLoRA fine-tuned Qwen2.5-1.5B-Instruct model from Assignment 1 is used only as the specialist classifier, which matches the task it was trained for. Its outputs, such as account_access, are normalized into the workflow's canonical labels. Deterministic Python rules then handle policy, deadlines, missing information, duplicates, risk and routing.

Figure 1. Agent workflow and conditional branches



This division keeps the useful language understanding of the model while making safety-critical behavior

predictable, testable and auditable. The classifier achieved 89.20% accuracy and 89.10% Macro F1 on 250 unseen examples.

3. Real and Mocked Tools

Real components: The fine-tuned Qwen classifier runs the actual complaint classification. The LangGraph orchestration, text normalization, output parser, validation, policy mapping, completeness checks, duplicate logic, SLA calculations, risk rules and routing logic are executable Python components rather than generated descriptions.

Mocked components: The customer database, previous-ticket database, ticket-creation backend, escalation queue, department notification system and audit-log storage use in-memory Python structures. They simulate the interfaces a production agent would call without connecting to real banking data. Each action records its input, output, timestamp and mocked status in the trace.

4. Demonstrated Behavior

The notebook demonstrates normal routing of an account-access complaint to Digital Banking Support, escalation of an urgent English-Roman Urdu fraud complaint to the Fraud Investigation Unit, ticket creation and duplicate-ticket reuse, structured trace logging, mocked backend activity, and rule-based separation of a complaint containing multiple issues. For multi-issue input, the prototype separates the issues but processes only the first issue through the full graph.

5. Current Limitations

- **Confidence and model errors:** The displayed confidence is an output-validity proxy, not a calibrated probability. A valid category string can still be semantically wrong, so classification errors require monitoring and human-review safeguards.
- **Prototype integrations:** All banking records and action systems are mocked in memory, reset when the notebook session restarts, and do not provide persistence, authentication, concurrency control or core-banking integration.
- **Policy scope:** The 48-hour acknowledgement, 10-working-day interim response, 30-day fraud response and 45-day external escalation values are simplified configurable demonstration rules. They require legal and compliance validation before production deployment.
- **Conversation coverage:** Only the first detected issue receives full processing, and customer acknowledgements are template-based. The current prototype does not yet conduct a complete multi-turn evidence-gathering conversation.
- **Next step:** Replace the mocked backends with authenticated banking services, calibrate model confidence, expand multilingual testing, and validate every policy with the bank's legal and compliance teams.

Kaggle NoteBook: https://www.kaggle.com/code/batoolbintefazal/complaint-intake-and-compliance-routing-agent
Recorded Demo Videi: https://drive.google.com/file/d/1DaqtgX9KaRqUh7a5pOHXQJY8azeBL5CG/view?usp=sharing
Github Repository: https://github.com/batoolfazal/Complaint-Intake-And-Compliance-Routing-Agent